

나머지 내용 정리

DTO VS Entity 분리

왜 Entity를 그대로 노출하면 안되는가

제 생각에는 Entity에는 데이터베이스 내용이 들어 있기 때문에

불필요한 노출을 줄이기 위해 DTO를 사용한다로 이해하였고,

예를 들어 패스워드나 주요 정보들이 entity에는 있지만 DTO에서는 주요정보를 제외한 정보들만 처리하는 구조로 이해했습니다.

위험 요소 5가지는 보안,순환참조 API불안정, 과다노출, N+1 문제가 있습니다.

보안 : 민감 필드가 응답에 섞임,

순환참조 : JPA 양방향 연관관계가 JSON 직렬화에서 무한 루프로

API 불안정은 DB 스키마 변경이 곧바로 API 응답 변경으로 이어짐

과다 노출 : 클라이언트가 필요 없는 필드까지 다 받음

N+1 문제 : Lazy 로딩 필드를 직렬화하다 추가 쿼리 폭발

권장패턴 – 레이어별 객체 사용하는 방법입니다.

Controller에서는 Request/Response DTO를 사용하고

Service에서는 Entity (도메인 객체)를 사용

Repository에 Entity를 사용합니다.

변환 도구 비교가 정확히 이해가 되지는 않습니다.

Transactional 어텐션은

Service 레이어에 뒤야하며 주로 읽기를 많이 사용하기 때문에 읽기전용으로 하

여 성능 최적화를 하고, 쓰기 메서드는 readOnly를 해제하여 작동하게 합니다.

클래스 레벨에서 사용하는 이유는 Hibranate는 readOnly 트랜잭션에서 dirty checking을 건너뛰어 성능이 향상됩니다.

읽기 메서드가 훨씬 많기 때문에 기본으로 readOnly에 두고 쓰기만 오버라이딩합니다.

절대 하지 말아야 할 것

1.Controller에서 transactional 어노테이션

응답 직렬화 동안 트랜잭션 유지를 하게되면 락 시간이 증가합니다.

2. 같은 클래스 내 자기 호출 금지

자기 자신을 호출하게되면 프록시를 거치지 않게 되고 그러면 트랜잭션이 무시됩니다.

(Spring은 @Transcational은 AOP (관점지향프로그래밍) 프록시 기반이라 객체 내 메서드 호출은 가로채지 못합니다.

3.private 메서드에 @ Transcational

프록시가 가로챌 수 없습니다. (무시됨)

4.checked exception에서 자동 롤백 안 됨

기본 동작: RuntimeException 만 롤백 checked exception에서도 롤백하려면:

@Transactional(rollbackFor = Exception.class

좋은 Spring 코드의 5가지 신호 정리

1. Controller가 얇다 — HTTP 변환 + 입력 검증만

2. Service가 도메인 언어로 작성됨 — cancelOrderAndRefund() 같이 비즈니스 의도가 메서드 이름에 드러남

3. 생성자 주입 + final 필드 — 의존성 명시, 불변성 보장

4. DTO와 Entity 분리 — API 안정성 + 보안 5. @Transactional이 Service 레이어

에만, readOnly가 기본 — 트랜잭션 경계 명확